

Verification Continuum™

IDX IP Debug Acceleration User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	4
Synopsys Statement on Inclusivity and Diversity	4

1. Introduction	6
------------------------------	----------

2. IDX IP Debug Acceleration Usage Model	7
Setting Up the Environment	7
The Flow	7
Preparing the Inputs	8
The Waveform (FSDB)	8
The VCS Compiled simv Executable for the IP Level Design	8
The Scope Mapping File	8
IDX IP Debug Acceleration Configuration File	8
Running IDX IP Map Compilation	10
Running IDX Waveform Replay Simulation	11
Executing an Example	11
The Design	11
Perform IP Map Compilation	12
Replaying SoC Waveform Stimulus to the IP Design	13
Limitations and Known Issues	13

Preface

IDX enables reusing waveforms to avoid continually re-creating and re-running stimulus. As the waveform (usually FSDB) records the values, each signal/bus contains throughout the simulation run, it is a good source to 'replay' the values as the stimulus in a simulation.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetsynopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to support_center@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to support_center@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

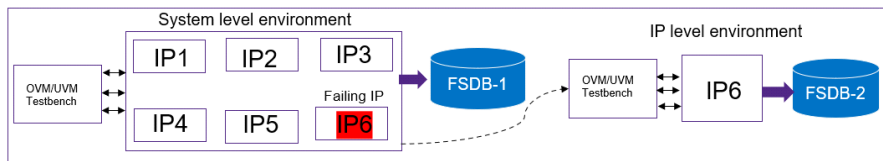
language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

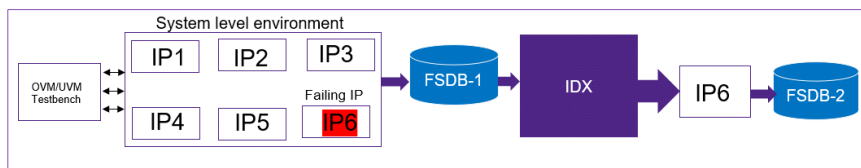
The pre-silicon verification involves running simulation at unit-level (or IP level), cluster-level, and finally at SoC level. The tests at different levels are usually different. When a simulation failure happens at SoC level, it requires a lot of iterations to create a test scenario to replicate the bug at all lower levels of validation, especially the IP level as shown in the following figure. This bug replication process is very challenging as an SoC environment is always very complex. In many cases it is possible the bugs cannot be reproduced at lower levels. As a result, the engineers must debug at SoC level directly. What is even worse, any context changes (a patch that contains the suspected fix of the bug) requires the re-compilation and re-simulation at SoC level, which is time consuming and inefficient.

Figure 1 *Reproduction of SoC Simulation Environment to IP level*



To resolve above problem, IDX enables reusing waveforms from SoC simulation to the IP environment as shown in the following figure. As the waveform (usually FSD) records the values each signal/bus contains throughout the simulation run, it is a good source to 'replay' the values as the stimulus in a simulation. The 'replayed' values can be applied to every level of validation, thus substantially save debug time and improve productivity. The flow that reuses an SoC waveform to replay the simulation scenario at IP level is referred to as 'IDX IP Debug Acceleration'.

Figure 2 *Replay SoC Simulation Waveform (FSD) to IP Level by IDX*



2

IDX IP Debug Acceleration Usage Model

The following sections describe how to enable IDX IP Debug Acceleration flow and gives a demo case to show how it works.

Setting Up the Environment

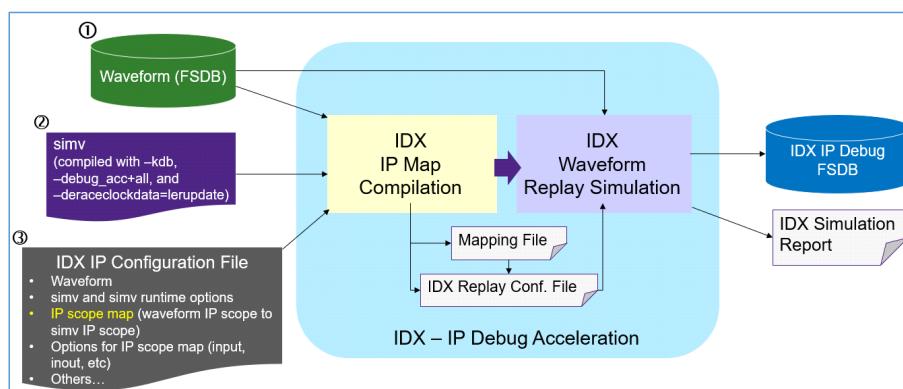
You need to run the following command in the C (CSH) shell environment before running IDX:

```
% setenv VERDI_HOME <verdi_installation_path>
% setenv PATH $VERDI_HOME/bin:$PATH
```

The Flow

The following figure illustrates the flow of IDX IP Debug Acceleration:

Figure 3 *IDX IP Debug Acceleration*



As the flow illustrates, IDX IP Debug Acceleration flow consists of two steps:

Step 1: IDX IP map compilation

Step 2: IDX waveform replay simulation

The detailed usage of above steps is described in the following sections.

To enable IDX runtime simulation, the following data needs to be prepared as the inputs.

Preparing the Inputs

1. The waveform (FSDB) for reuse
2. The VCS compiled `simv` executable for simulation
3. The mapping file which specifies the signal names to be 'replayed' from the waveform to the `simv`
4. The IDX configuration file

The Waveform (FSDB)

IDX allows to read the waveforms, that is, FSDB generated by any of the tools.

The VCS Compiled `simv` Executable for the IP Level Design

The following VCS compilation options must be added:

- `-debug_acc+f+w+cbk+fwn` (or `-debug_acc=all`)
- `-kdb`

The following VCS compilation options are optional but suggested to be added:

`-deraceclockdata=lerupdate`

The option `-deraceclockdata=lerupdate` can resolve most of the race conditions during the IDX waveform replay simulation.

The Scope Mapping File

The scope mapping between the SoC hierarchy in the FSDB and the IP hierarchy in the `simv` executable must be provided.

The following is an example of the scope mapping file used in IDX. The left hand side of the symbol '`=>`' is the IP level full hierarchical name in the `simv` executable, while the right hand side is the SoC level full hierarchical name in the FSDB.

```
// <simv_scope_full_name> => <waveform_scope_full_name>
wrap.u_cpu_ip => testbench.soc.dut_wrap.dut.cpu_ip_01
```

IDX IP Debug Acceleration Configuration File

To put all above necessary inputs together, IDX requires a IP Debug Acceleration configuration file. The following example shows the format of the configuration file:

```
#####
# Common fields
```

```
# The field here will give impact on all waveform files
#####

//simv.daidir abs. path. [MUST HAVE] (Optional in simv. first usage)
SIMV_DAIDIR = $PWD/simv.daidir

//necessary runtime options for running original simulation. (Optional)
SIMV_OPT = -l simv_eth.log

//waveform dumping Tcl commands. Use semicolon to separate tasks
(Optional)
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all; fsdbDumpSVA

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
only #####

##### record 1 #####
{
//waveform file abs. path. [MUST HAVE]
WAVEFORM = $PWD/novas.fsdb

//specify the IP scope mapping between waveform and the simv. [MUST HAVE]
SCOPE_MAP =    scope_map_file
AUTO_MAP =    +in +inout
}

##### record 2 #####
{
//can be empty as a placeholder
}
```

In general, the configuration file inherits all fields and settings of the configuration file used in IDX 'waveform replay' simulation. The only difference is, in the record field, instead of specifying the mapping file, the `SCOPE_MAP` and `AUTO_MAP` are used.

The `SCOPE_MAP` describes the scope mapping file, see [The Scope Mapping File](#)). The `AUTO_MAP` can be one or many of the following:

Attribute	Description
+in	Maps the input ports of the scope. For example, <code>AUTO_MAP = +in</code>
+out	Maps the output ports of the scope. For example, <code>AUTO_MAP = +out</code>
+inout	Maps the inout ports of the scope. For example, <code>AUTO_MAP = +inout</code>

Attribute	Description
+mda	Maps the multi-dimensional array signals under the scope. For example, <code>AUTO_MAP = +mda</code>
+reg	Maps the registers under the scope. For example, <code>AUTO_MAP = +reg</code>

The `AUTO_MAP` setting can use multiple attributes, for example:

```
...
AUTO_MAP = +in +inout
...
```

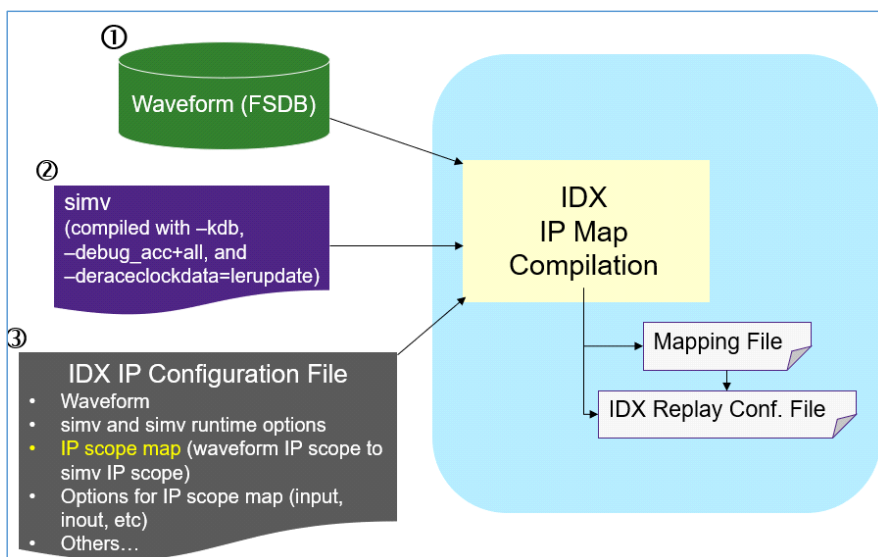
Running IDX IP Map Compilation

Perform the following command to run IDX IP map compilation:

```
% idx -compile_map <IP_map_compilation_config_file> [-o
<ip_map_directory_name>]
```

The command executes the step as shown in the following illustration:

Figure 4 IDX IP Debug Acceleration



If the option `-o <ip_map_directory_name>` is not specified, the default output directory `idx.map.dbdir/` is generated under the current path.

After finishing the step, the IDX gives the following messages:

```
[IDX_INFO]Generate new config file '<ip_map_directory>/idx.cfg' begin.  
[IDX_INFO]Generate new config file end.  
[IDX_INFO]Please use the command below to run replay process:  
> idx -cfg <ip_map_directory>/idx.cfg -work WORK_DIR_NAME_YOU_WANT  
[IDX_INFO]Compile MAP done.
```

The message instructs how to proceed the follow-up IDX waveform replay simulation.

Running IDX Waveform Replay Simulation

After finishing IDX IP map compilation described in the [Running IDX IP Map Compilation](#) section, all the necessary files are available and described in the generated IDX runtime configuration file. Therefore, just follow the message to perform the IDX waveform replay simulation, such as:

```
% idx -cfg ./idx.map.dbdir/idx.cfg -work replay_work
```

Executing an Example

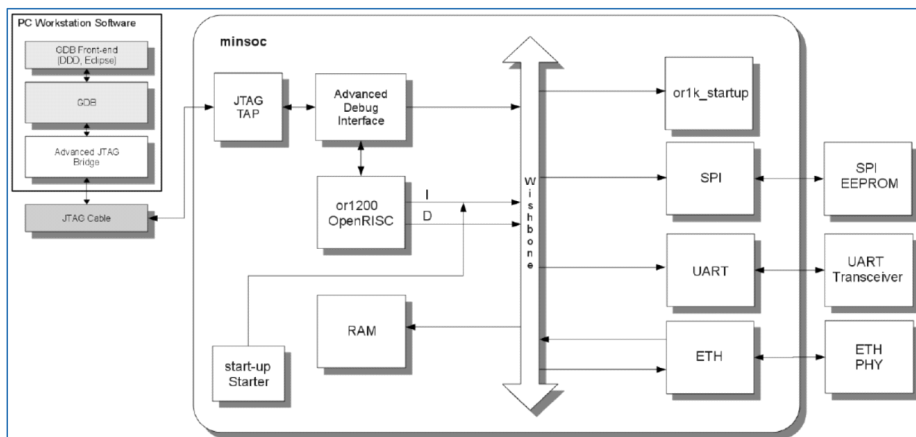
The Design

You need to use the MinSoC (Minimal OpenRISC System on Chip) design as an example to run IDX IP Debug Acceleration. The design case can be found in the following path:

```
$VERDI_HOME/demo/IDX/IP_Debug/
```

The MinSoC architecture is shown in the following figure:

Figure 5 MinSoC Architecture



The entire design hierarchy including the Testbench is illustrated as follows.

Figure 6 MinSoC Simulation Hierarchy

Hierarchy	Module
minsoc_tb_top	minsoc_tb_top
minsoc_top_0	minsoc_top
clk_adjust	minsoc_clock_manager
dbg_top	adbg_top
ethmac	ethmac
ethreg1	eth_registers
maccontrol1	eth_maccontrol
macstatus1	eth_macstatus
miim1	eth_miim
rxethmac1	eth_rxethmac
txethmac1	eth_txethmac
wishbone	eth_wishbone
onchip_ram_top	minsoc_memory_model
or1200_top	or1200_top
tap_top	TAP_TOP(ARCH)
tc_top	minsoc_tc_top
uart_top	uart_top
minsoc_if	minsoc_io
init_fpga_memory	

The `simv_ethmac` executable does not contain any stimulus to drive the IP `ethmac`. The Testbench module `test_ethmac` is just a wrapper to instantiate the IP.

Perform IP Map Compilation

To perform IP Map Compilation, go to the `idx_ip_work/` directory and execute the following command to perform IDX IP map compilation. You need to copy `$VERDI_HOME/demo/IDX/IP_Debug` to a directory `idx_ip_work` in your workspace.

```
% cd idx_ip_work
% idx -compile_map compile_ip_map.cfg
```

The configuration file `compile_ip_map.cfg` is shown as follows. You should use `'+in +inout'` attributes in the `'AUTO_MAP'` setting.

```
#####
# Common fields
# The field here will give impact on all waveform files
#####
SIMV_DAIDIR = $ROOT/ethmac_ip/simv_ethmac.daidir
SIMV_OPT    = -l ethmac.log +fsdbfile+ethmac_replay.fsdb
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all;

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
# only
#####
```

```
##### record 1 #####
{
WAVEFORM  = $ROOT/sim/soc.fsdb
SCOPE_MAP = ./ethmac_to_soc_scope_map.txt
AUTO_MAP  = +in +inout
}
```

The scope mapping file, `ethmac_to_soc_scope_map.txt`, used in the `SCOPE_MAP` setting is shown in the following example:

```
test_ethmac.u_ethmac => minsoc_tb_top.minsoc_top_0.ethmac
```

Replaying SoC Waveform Stimulus to the IP Design

In the `idx_ip_work/` directory, run the following command to perform IDX waveform replay simulation:

```
% make run_replay
```

Or

```
% idx -cfg idx.map.dbdir/idx.cfg -work replay_ETHMAC_work
```

After the IDX simulation, the replayed FSDB `ethmac_replay.fsdb` and the simulation log file `ethmac.log` are generated. We can begin the IP level debug with these files.

Limitations and Known Issues

The limitation of the IDX IP debug acceleration flow is listed as follows:

- IDX IP map compilation does not support Mixed-language/VHDL design.